

AWS re:Invent

NOV. 28 – DEC. 2, 2022 | LAS VEGAS, NV

NET201-R

Build highly available applications using Elastic Load Balancing

James Wenzel (he/him)

Principal Solutions Architect
AWS

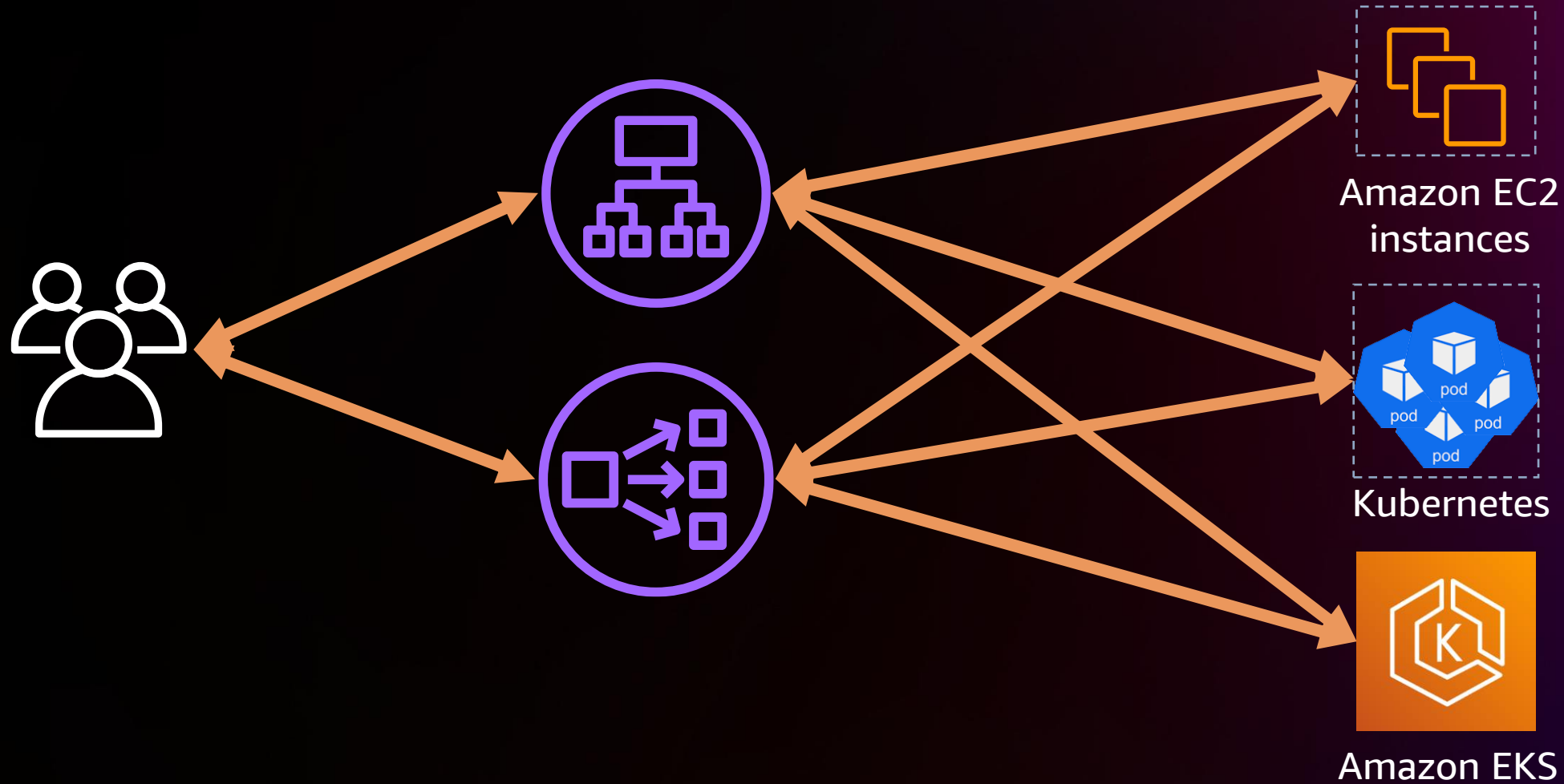
Ikenna Izugbokwe (he/him)

Principal Solutions Architect
AWS

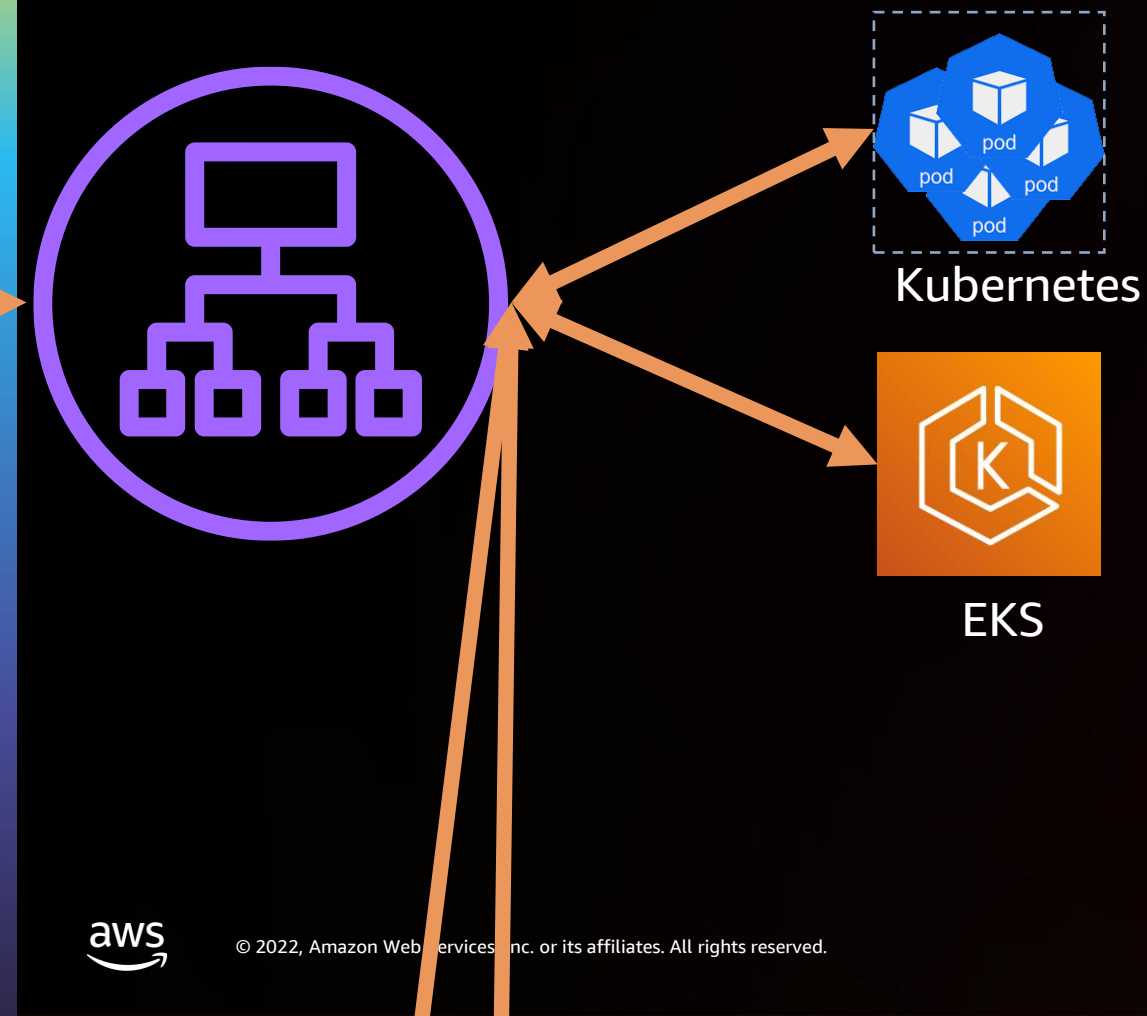


© 2022, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Elastic Load Balancing EC2 packaging



Kubernetes load balancer provisioning options



AWS cloud provider load balancer controller
vs.
AWS Load Balancer Controller

Related sessions

Session	Title	Time/location (Verify in App)
NET204	Application Networking Foundations	WEN-Level 1, Encore Ballroom 5 Dec. 1, 12:30 pm
NET212	Application Networking Best Practices	MGM-Level 3, Room 301 Dec. 1, 3:30 pm
NET320	Building your own cloud services using ELB Gateway Load Balancer	CSR-Level 1, Summit 217 Nov. 29, 12:30 pm
CON326	Are you ready? Essential Strategies for Kubernetes adoption	MND, Jasmine F Nov. 30, 12:30 pm
CON330	Operating Kubernetes clusters at cloud scale	MND, Islander F Nov. 30, 7:45 pm



Thank you!



Agenda

Application Load Balancer and Network Load Balancer

Application packaging

Infrastructure scaling

Availability dimensions

Application Load Balancer and Network Load Balancer



Benefits of using Elastic Load Balancers



Scalability

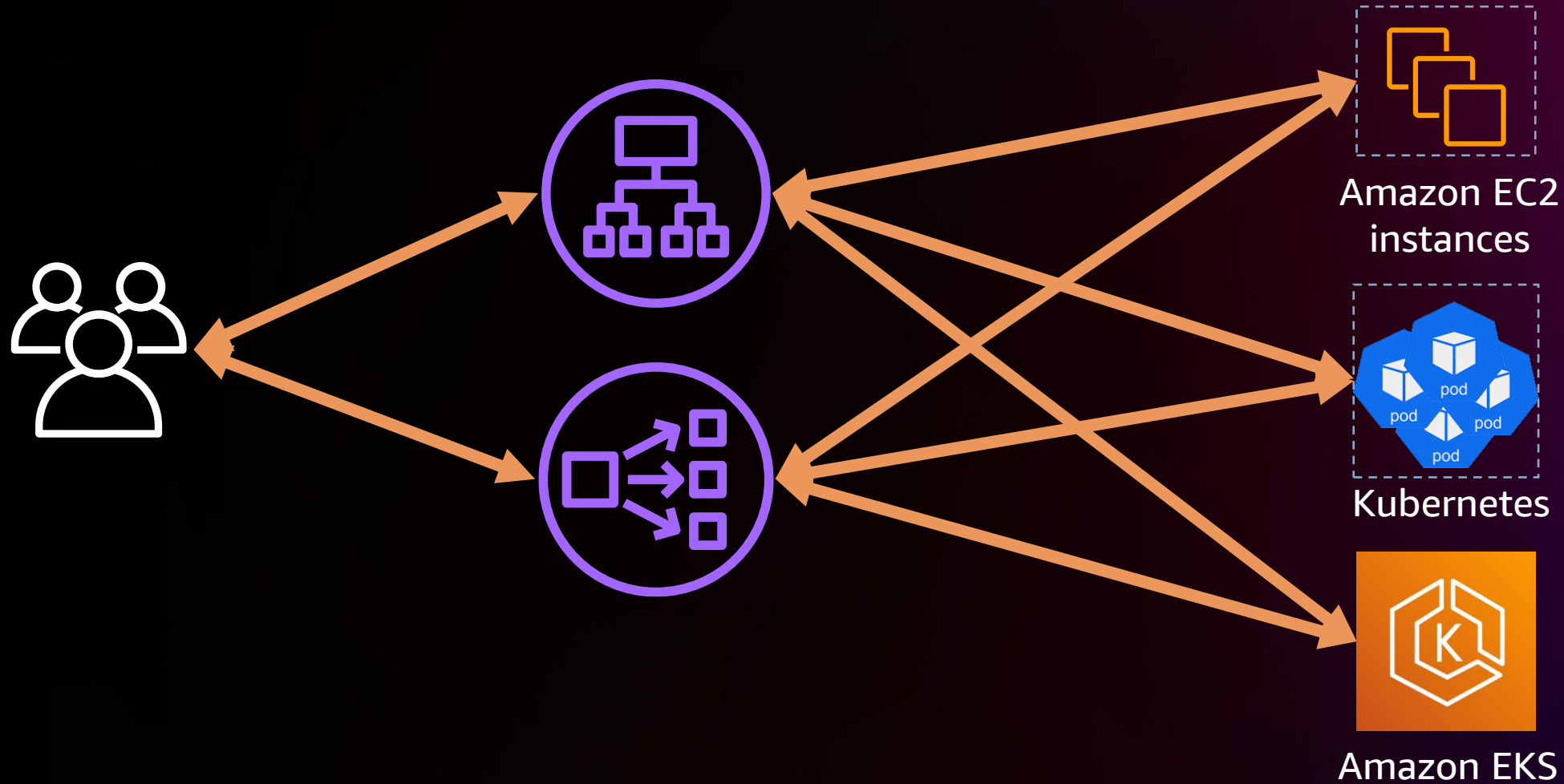


Security

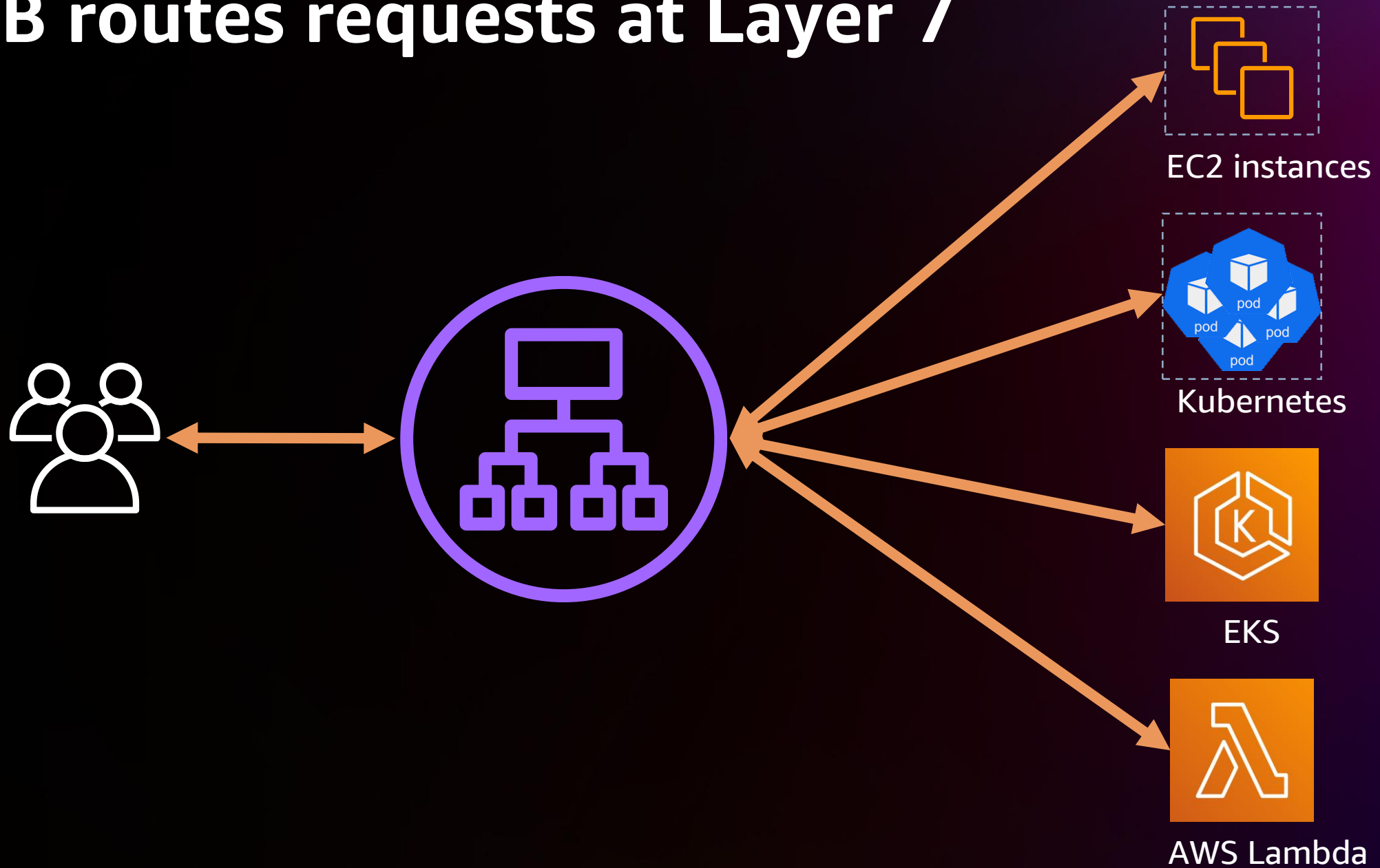


Availability

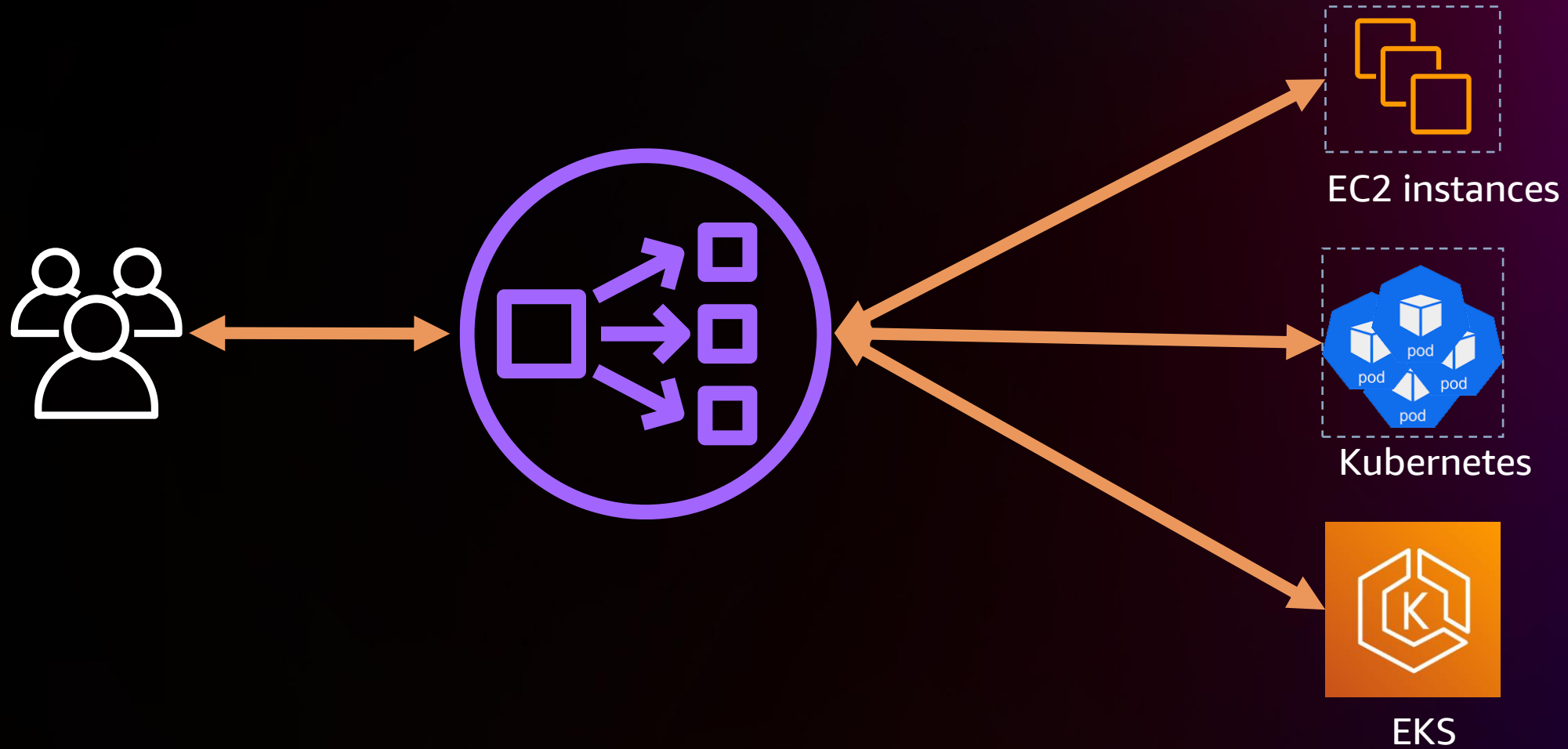
ELBs distribute incoming traffic to targets



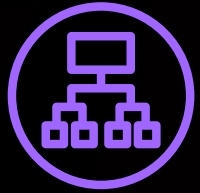
ALB routes requests at Layer 7



NLBs route TCP or UDP flows

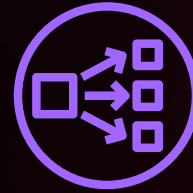


Overview of Elastic Load Balancers (ALB & NLB)



Application Load Balancer

- Request-level balancing
- HTTP, HTTPS, and gRPC
- Advanced rule-based routing
- Route direct to application servers



Network Load Balancer

- Connection (flow) level balancing
- TCP, TLS, and UDP support
- Built for scale, low latency, and high performance



Takeaways

- ELBs distribute load from clients to your application servers
- Traffic flows from your clients to ELB and then to your application
- ALB and NLB operate at different layers of the OSI model (Layer 4 and Layer 7)
- You choose load balancer based on traffic type your application uses

Application packaging



Best practices and takeaways

Provision with AWS Load Balancer Controller

Review AWS Load Balancer Controller service annotations

Leverage IP Targets when possible

Leverage tagging where needed

Provisioning details

AWS cloud provider load balancer controller

- Functionality provided by the AWS cloud controller manager
- Reconciles only Kubernetes Services resources of type LoadBalancer
- Environment and annotations define NLB implementation
- Deploys AWS CLB (default) and NLB in instance mode

AWS Load Balancer Controller (Recommended)

- Reconciles Kubernetes Services resources of type LoadBalancer and Ingress
- Formerly AWS ALB Ingress Controller
- Deploys NLBs in instance & IP mode for service resources
- Deploys ALB in instance & IP mode for ingress resources

Infrastructure scaling



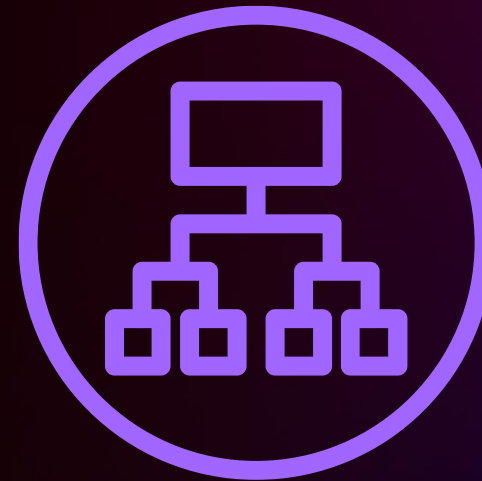
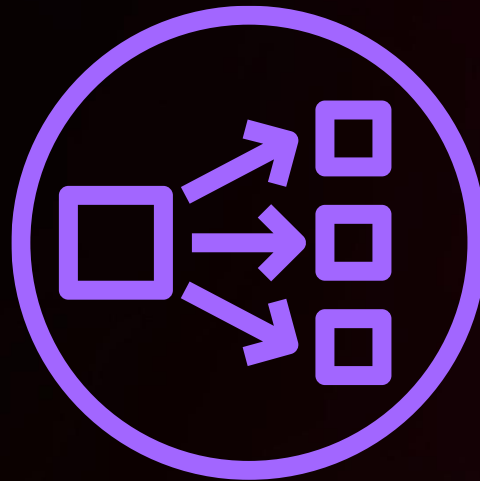
Scalability with ELB and EC2 instances

ELBs automatically scale to support virtually any workload

ELB scales up aggressively, and scales in cautiously

ALB scaling changes IPs

NLB scaling doesn't change IPs

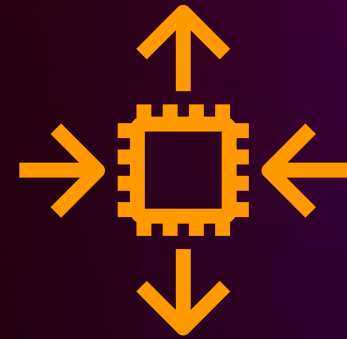


AWS Auto Scaling

AWS Auto Scaling monitors your applications and automatically adjusts capacity to maintain steady, predictable performance at the lowest possible cost

Using AWS Auto Scaling, it's easy to set up application scaling for multiple resources across multiple services in minutes

- You should scale on most critical metric
 - RequestCountPerTarget
 - Don't be afraid of making your own combined metric to scale on
- Align EC2 instance size with scaling units



Auto scaling

AWS Lambda

AWS Lambda is a serverless, event-driven compute service that lets you run code for virtually any type of application or backend service without provisioning or managing servers

You can trigger Lambda from over 200 AWS services and software as a service (SaaS) applications, and pay only for what you use

- Serverless
- Be aware of concurrency limits
- Lambda automatically scales; make sure your dependencies do too



AWS Lambda

Scaling Kubernetes/EKS clusters



Kubernetes Cluster Autoscaler

The Karpenter logo, featuring a stylized '@' symbol with a house icon inside, followed by the word 'Karpenter' in a bold, sans-serif font. The background is a light blue square with various construction tools like a ruler, pencil, and paintbrush scattered around.

@Karpenter

Just-in-time Nodes for Any
Kubernetes Cluster

Kubernetes Autoscaling highlights

- Kubernetes Cluster Autoscaler
 - Uses Amazon EC2 Auto Scaling groups (ASGs) in AWS
 - Auto Scaling groups require tagging
 - Scope node groups across more than one Availability Zone
 - Cluster Autoscaler can balance the scaling of the Amazon EC2 Auto Scaling groups
- Karpenter
 - Observes incoming pods over cluster lifetime
 - Can bypass Kubernetes scheduler and work with compute provider (EC2)
 - Designed to be cloud agnostic
 - Manages instances directly (group-less node provisioning)

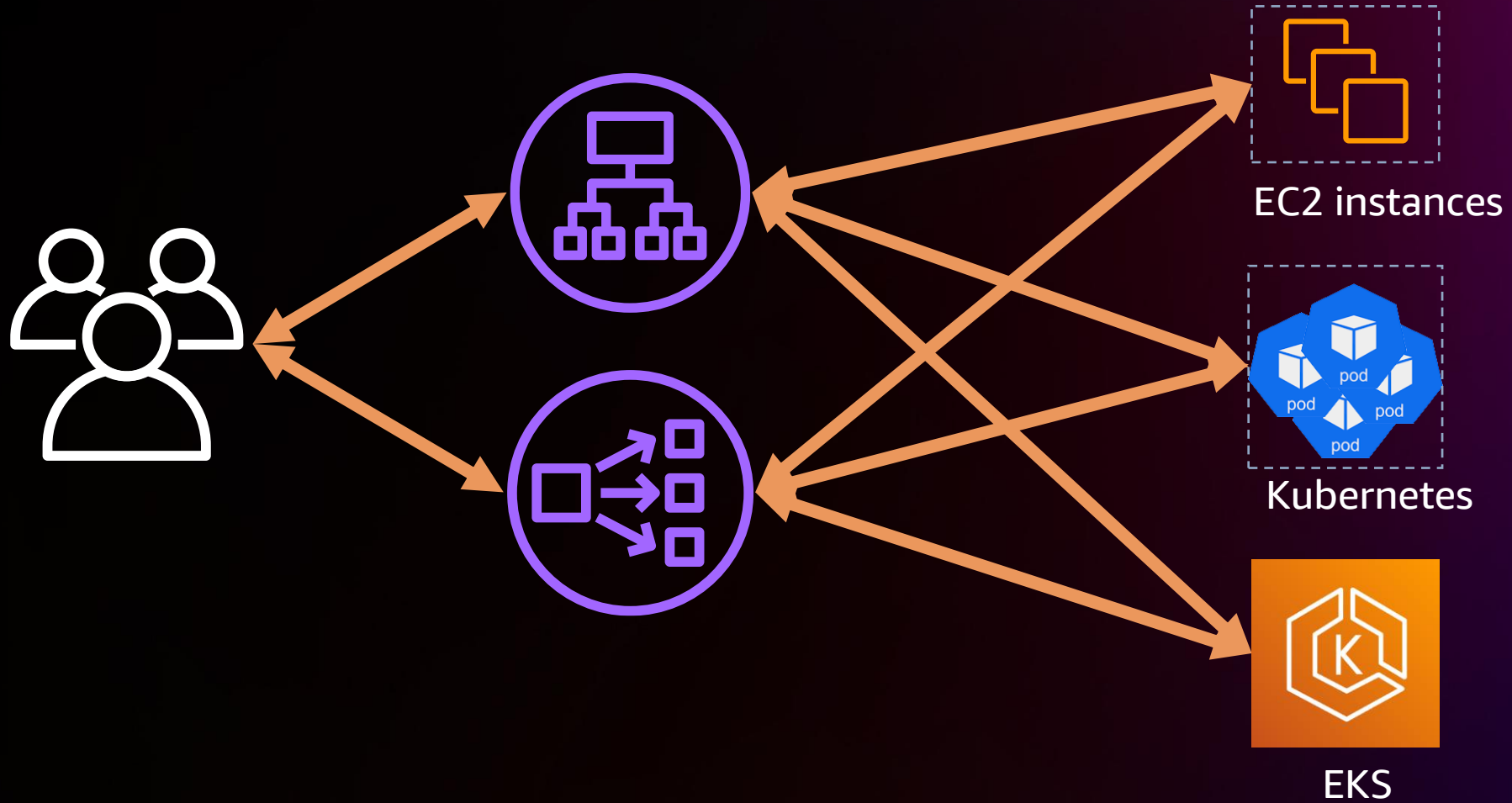
Best practices and takeaways

- Create multiple node groups when using Cluster Auto Scaler
 - Ensure matching labels for Cluster Auto Scaler Balancing
 - Use managed node groups with the Cluster Autoscaler (if possible)
 - Consider Karpenter for additional scaling flexibility
-
- ELBs automatically scale to virtually any workload
 - You should scale on most critical metric
 - RequestCountPerTarget
 - Don't be afraid of making your own combined metric to scale on
 - Align EC2 instance size with scaling units
 - Lambda automatically scales; make sure your dependencies do too

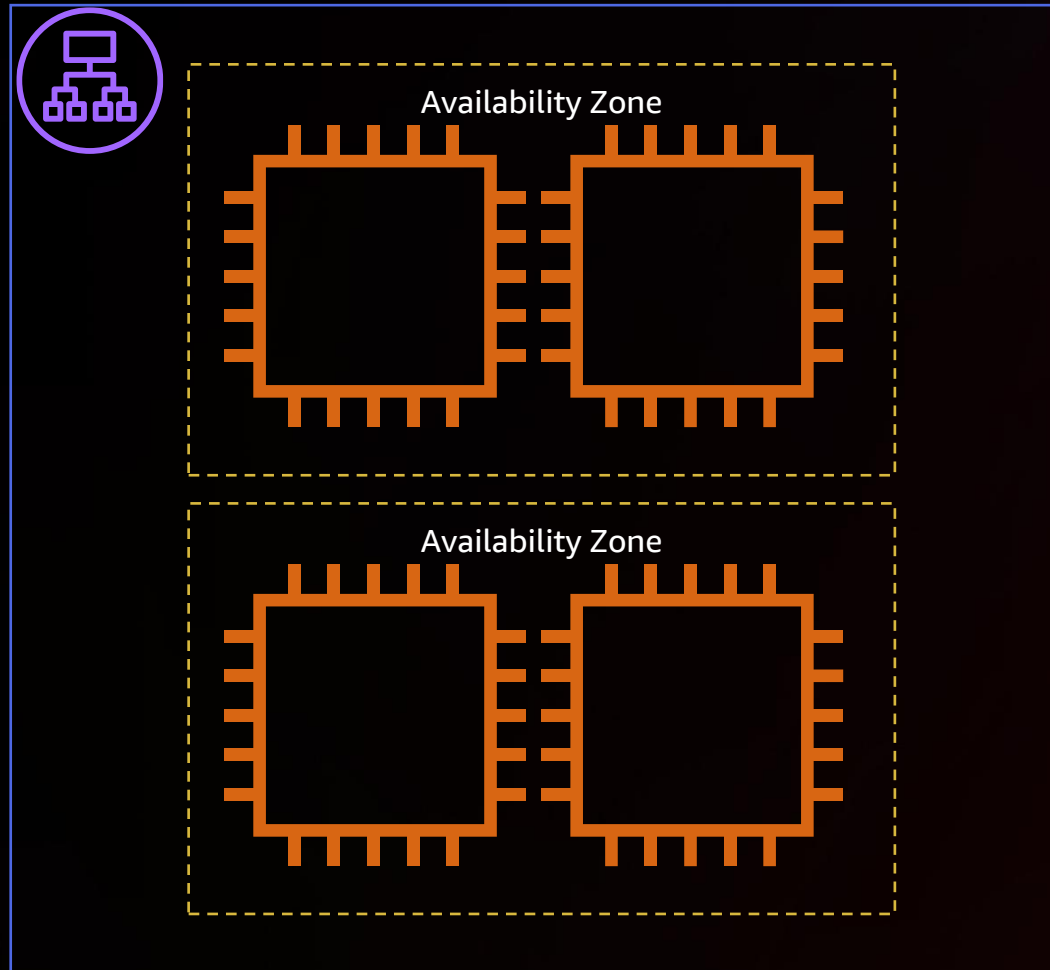
Availability dimensions



Elastic Load Balancers improve availability

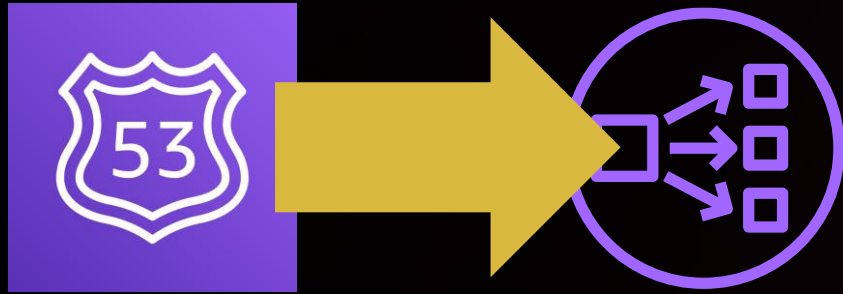


How we do it – Availability

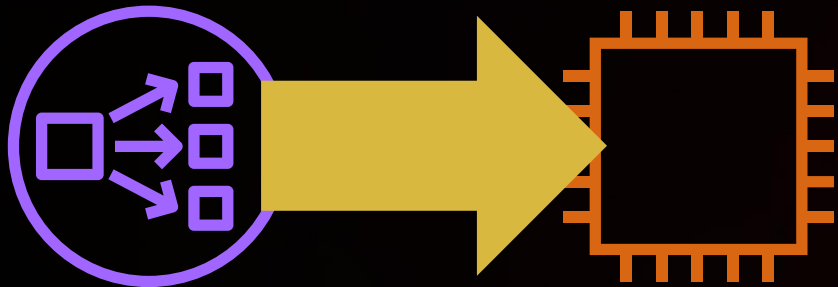


- Over provision 1AZ
- Health check – remove IP
- Health check replacement
- Anomaly detection
- Fleet lifecycle systems

ELBs have 2 levels of health checks



Route 53 health checks each ELB IP
(per AZ on NLB)



Each IP/AZ health checks all targets
that it could route traffic to

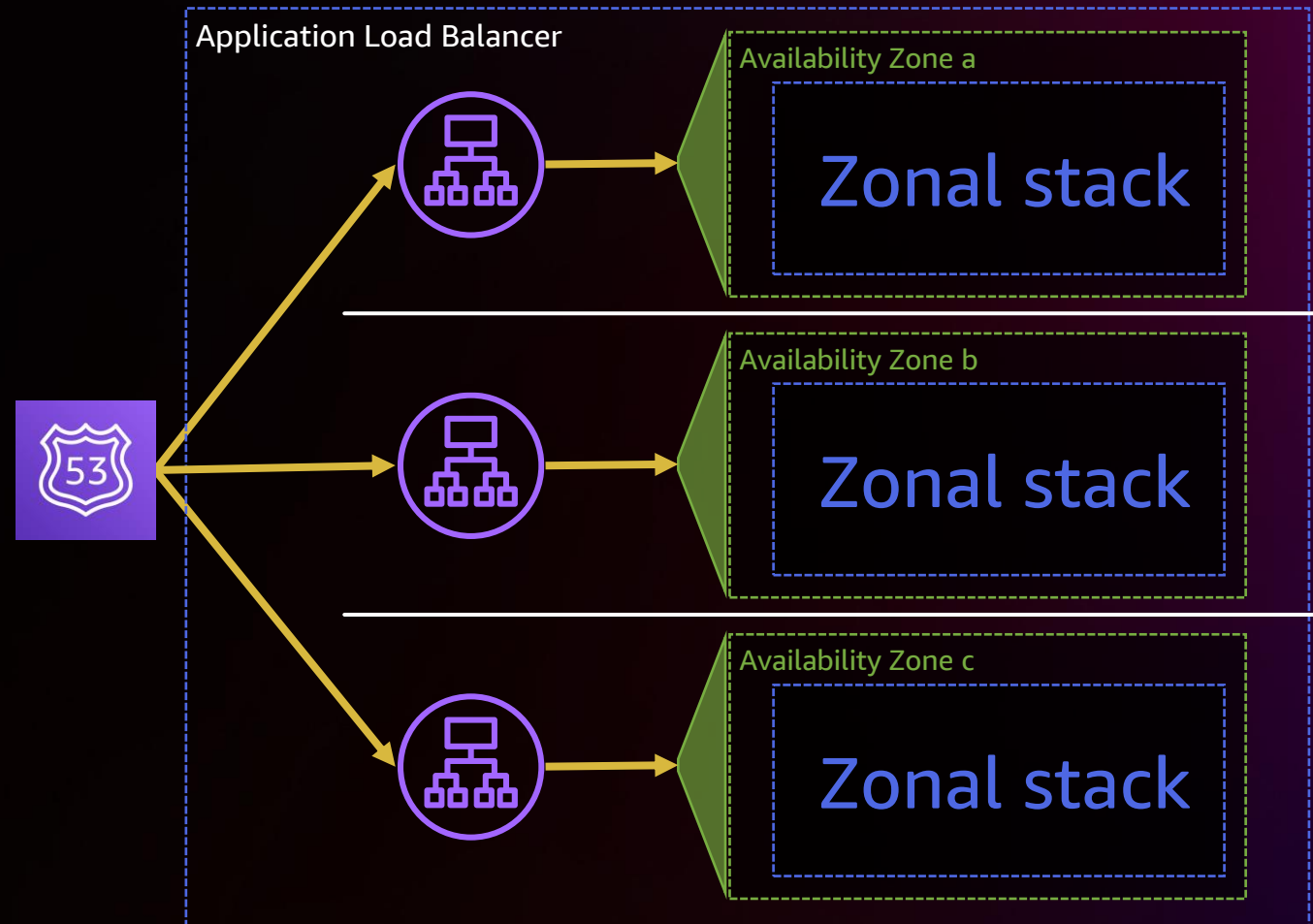
(NLB with cross zone – all targets have independent
health checks from each AZ)

ALB turn off cross zone load balancing

Requests stay in the AZ they arrived in.
Remove AZ from DNS will remove the targets.

This is set at the target group level.

When an AZ is down, Amazon Route 53 automatically removes it from DNS. With CZ off, targets in that AZ will not receive traffic.



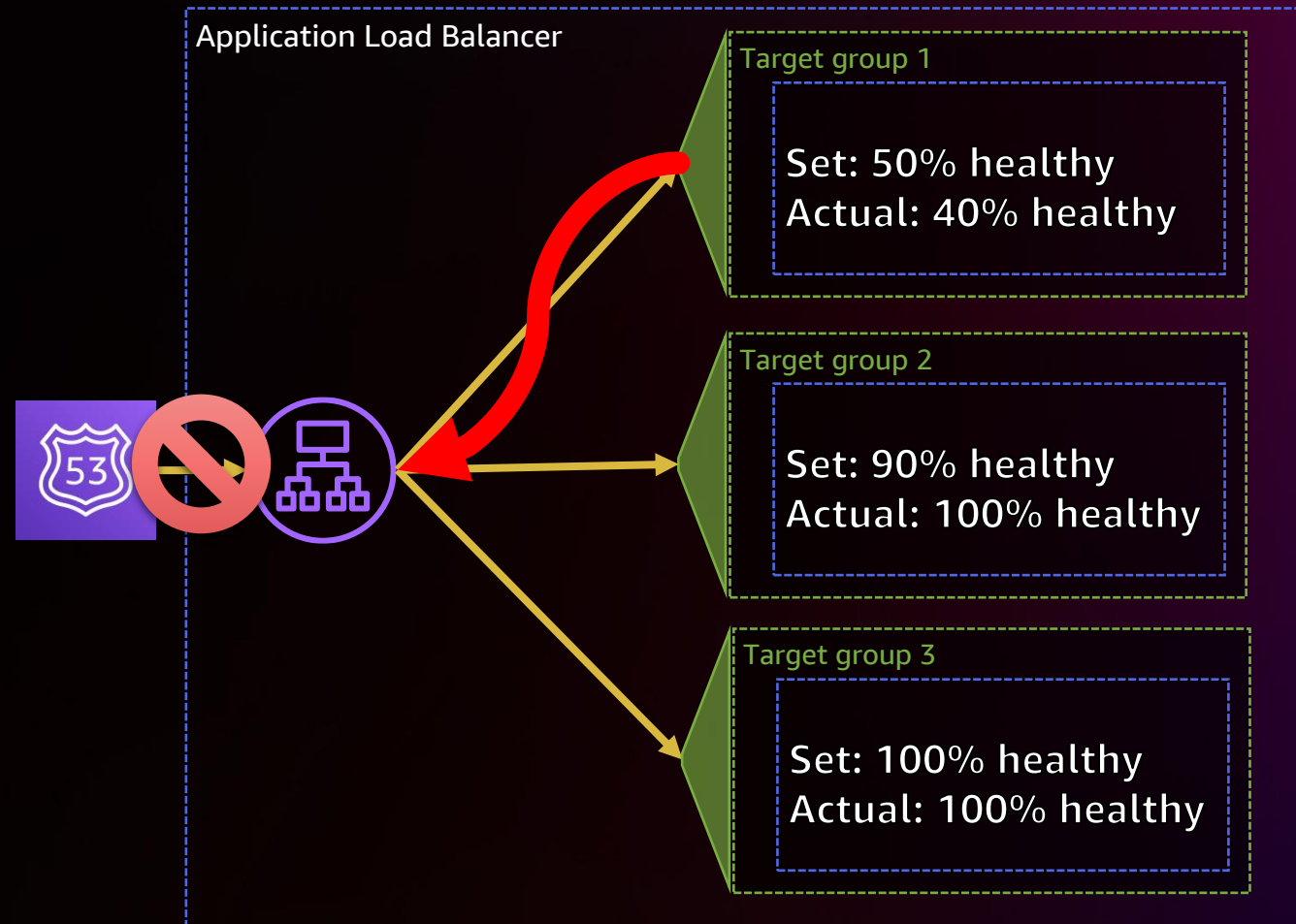
Target group health – DNS failover

Change threshold when to fail Amazon Route 53 DNS health check.

Default 100% unhealthy.

Set a number of targets or percent. Set both, we use both.

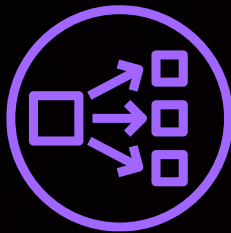
Must be less healthy than fail open setting to targets in the same target group.



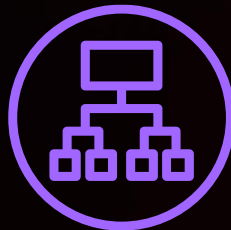
Availability with ELB and EC2 instances



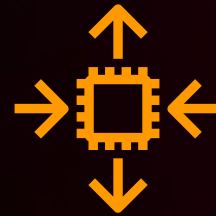
Route 53



NLB



ALB



Auto scaling



EC2 instances



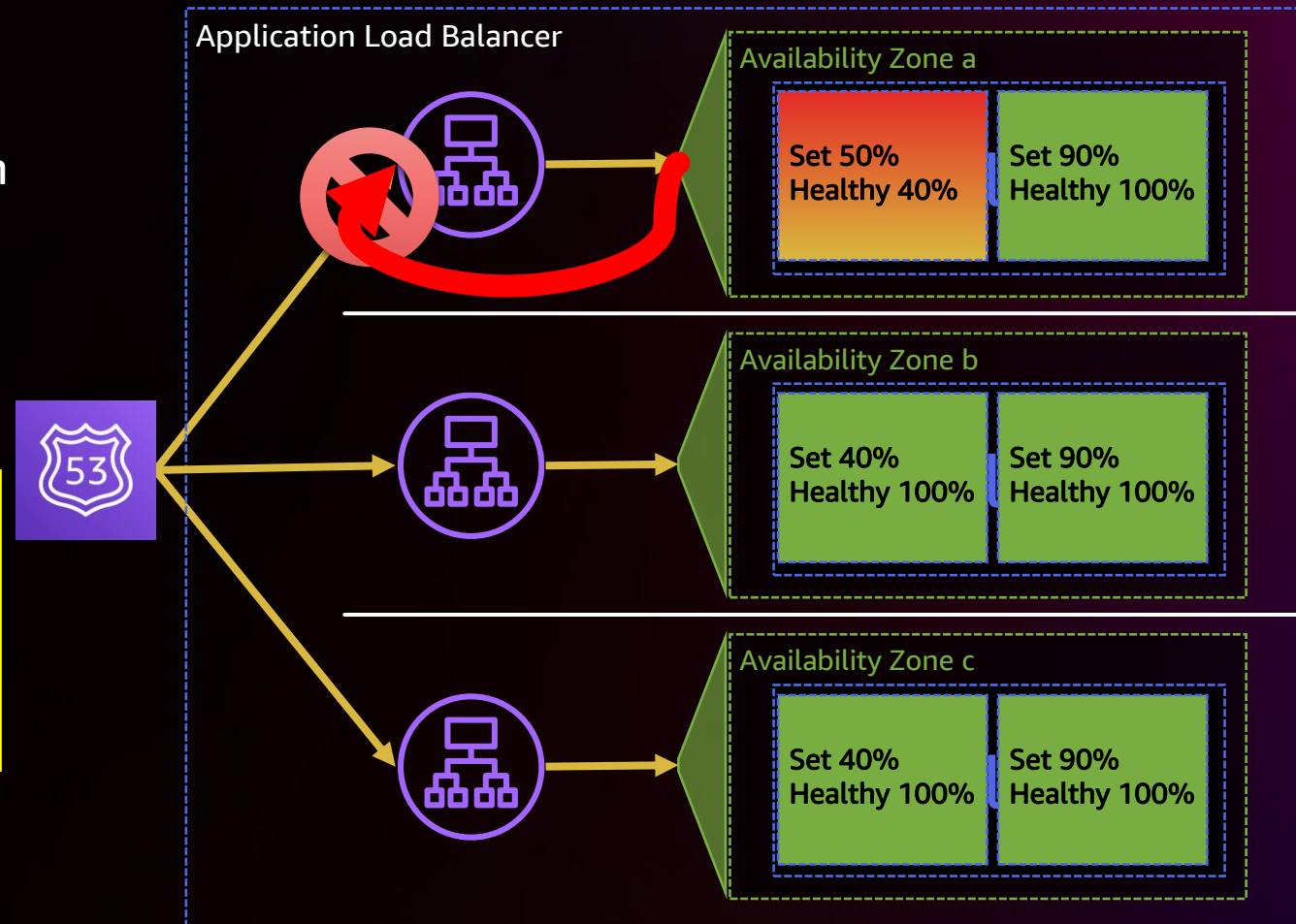
AWS Lambda

Target group health – DNS failover

With Cross Zone turned off you can now fail away faster from an impaired AZ.

You can **also** turn off DNS failover for a target group.

Useful for preventing fail-away for non-critical target groups.



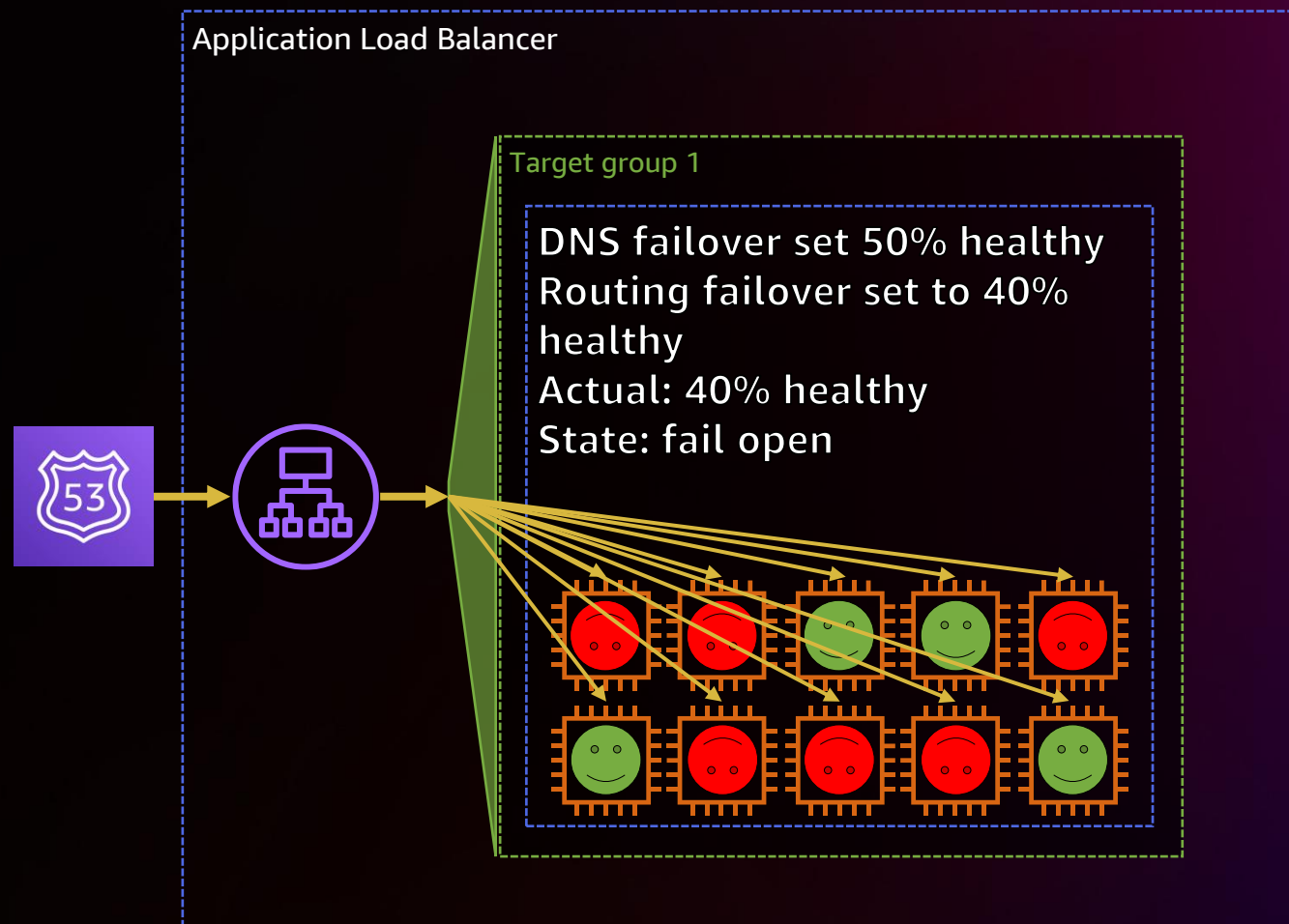
Target group health – routing failover

Change threshold when to fail open.

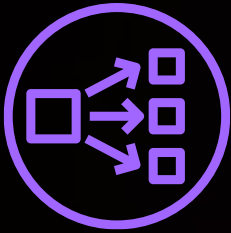
Fail open treats all targets, healthy or unhealthy, as if they were healthy.

Default 100% unhealthy.

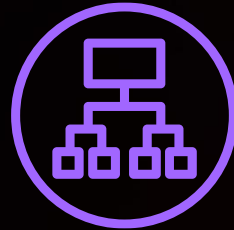
Must be more healthy than DNS failover setting the same target group.



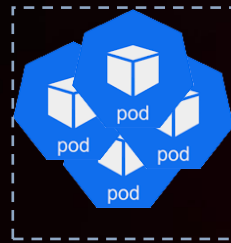
Availability with ELB and Kubernetes/EKS



NLB



ALB



Kubernetes



EKS

Kubernetes availability recommendations

Utilize Pod readiness gates

Account for ELB deregistration delay

Availability best practices and takeaways

With Kubernetes you can improve availability:

- Utilize Pod readiness gates
- Account for ELB deregistration delay

When you use an ELB you automatically get:

- Health checks – Route 53 and targets
- Automatic handling of failures
- Integrations

Lambda is highly available serverless

- Ensure dependencies are available

Where can I learn more?

Load Balancer Controller

<https://kubernetes-sigs.github.io/aws-load-balancer-controller/v2.4/>

Karpenter

<https://karpenter.sh/>

Managed Node Groups

<https://docs.aws.amazon.com/eks/latest/userguide/managed-node-groups.html>

